Security Incident Reporting Portal

Project Documentation & Developer Manual

---

1. Project Overview

The **Security Incident Reporting Portal** is a lightweight, defensive security (Blue Teaming) application built with Python, Flask, and SQLite. It serves as a centralized intake system where organization members can quickly report suspicious security events—such as phishing emails, lost hardware, unauthorized account logins, or potential malware infections.

Reports submitted through the portal are automatically assigned a unique Incident Reference ID (e.g., `INC-49201`), timestamped, categorized by perceived severity, and populated into an interactive **Security Operations Center (SOC) Triage Dashboard** for real-time tracking and remediation.

---

2. System Architecture & Components

The application follows a clean, single-file MVC architecture operating within a lightweight Flask application context:

```
None
+-------------------------------------------------------------------+
|                        EMPLOYEE / REPORTER                        |
+-------------------------------------------------------------------+
                                  |
                                  v  HTTP POST (/report)
+-------------------------------------------------------------------+
|                     FLASK CONTROLLER (app.py)                     |
|  - Generates Unique Incident Ref (INC-XXXXX)                      |
|  - Appends System Timestamps                                      |
|  - Validates and Sanitizes Form Input                             |
+-------------------------------------------------------------------+
                                  |
                                  v  SQL INSERT
+-------------------------------------------------------------------+
|                    SQLITE DATABASE (incidents.db)                 |
|  - Table: `incidents`                                             |
|  - Columns: id, ref, reporter, category, severity, status, date   |
+-------------------------------------------------------------------+
```

```
                                |
                                v  SQL SELECT (ORDER BY id DESC)
+-------------------------------------------------------------------+
|                    SOC ANALYST TRIAGE DASHBOARD                   |
|  - Real-time Incident Table                                       |
|  - Severity Badging (LOW, MEDIUM, HIGH, CRITICAL)                 |
+-------------------------------------------------------------------+
```

Key Components:

1. **Intake Form Interface:** Captures user identity (`email`), threat taxonomy (`category`), urgency (`severity`), and descriptive details.
2. **Database Engine:** An embedded SQLite database (`incidents.db`) that initializes schema tables automatically on application launch.
3. **Reference ID Generator:** Uses timestamp hashing to construct collision-resistant tracking tags (`INC-<timestamp>`).
4. **SOC Triage Dashboard:** Renders incident records in reverse chronological order with color-coded severity tags.

---

3. Incident Taxonomy & Severity Matrix

The portal categorizes incoming events into four severity tiers to help SOC analysts prioritize incoming tickets:

Severity Level	Badge Color	Category Examples	Description & Impact
LOW	Green (`#22c55e`)	General Inquiries, Informational	Minor suspicious activity, low risk inquiries, spam.
MEDIUM	Yellow (`#eab308`)	Phishing, Single Device Loss	Unconfirmed phishing clicks, non-critical workstation issues.
HIGH	Orange (`#f97316`)	Active Malware, Compromised Creds	Confirmed credential theft, active virus alerts, stolen laptops.
CRITICAL	Red (`#ef4444`)	Data Breach, System Outage	Ransomware outbreaks, unauthorized DB access, exfiltration.

---

4. Complete Application Source Code (`app.py`)

Python

```
import sqlite3
import datetime
from flask import Flask, request, redirect, url_for, render_template_string

app = Flask(__name__)
DB_FILE = "incidents.db"

# 1. Database Initialization
def init_db():
    conn = sqlite3.connect(DB_FILE)
    cursor = conn.cursor()
    cursor.execute('''
        CREATE TABLE IF NOT EXISTS incidents (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            incident_ref TEXT NOT NULL,
            reporter_email TEXT NOT NULL,
            category TEXT NOT NULL,
            severity TEXT NOT NULL,
            description TEXT NOT NULL,
            status TEXT DEFAULT 'NEW',
            created_at TEXT NOT NULL
        )
    ''')
    conn.commit()
    conn.close()

init_db()

# 2. HTML UI Template (Form + SOC Dashboard)
HTML_TEMPLATE = """
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Security Incident Reporting Portal</title>
    <style>
        body { font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto,
sans-serif; margin: 40px; background-color: #0f172a; color: #f8fafc; }
        .container { max-width: 900px; margin: 0 auto; }
        .card { background-color: #1e293b; padding: 24px; border-radius: 10px;
margin-bottom: 24px; border: 1px solid #334155; }
        h1, h2 { color: #38bdf8; }
        label { display: block; margin-top: 10px; font-weight: bold; }
```

```
        input, select, textarea { width: 100%; padding: 10px; margin-top: 5px;
border-radius: 6px; border: 1px solid #475569; background-color: #0f172a; color:
white; box-sizing: border-box; }
        button { background-color: #0284c7; color: white; border: none; padding:
12px 20px; border-radius: 6px; margin-top: 15px; cursor: pointer; font-weight:
bold; }
        button:hover { background-color: #0369a1; }
        table { width: 100%; border-collapse: collapse; margin-top: 15px; }
        th, td { border: 1px solid #334155; padding: 12px; text-align: left; }
        th { background-color: #0f172a; color: #38bdf8; }
        .badge { padding: 4px 8px; border-radius: 4px; font-weight: bold;
font-size: 0.85em; }
        .CRITICAL { background-color: #ef4444; color: white; }
        .HIGH { background-color: #f97316; color: white; }
        .MEDIUM { background-color: #eab308; color: black; }
        .LOW { background-color: #22c55e; color: white; }
    </style>
</head>
<body>
    <div class="container">
        <h1>🛡️ Security Incident Reporting Portal</h1>

        <!-- Incident Reporting Form -->
        <div class="card">
            <h2>Report a Security Incident</h2>
            <form action="/report" method="POST">
                <label>Your Email Address:</label>
                <input type="email" name="email" required
placeholder="user@company.com">

                <label>Incident Category:</label>
                <select name="category">
                    <option value="Phishing Email">Phishing Email / Suspicious
Link</option>
                    <option value="Lost / Stolen Device">Lost / Stolen Laptop or
Mobile</option>
                    <option value="Unauthorized Access">Unauthorized Account
Access</option>
                    <option value="Data Leak">Accidental Data Leak /
Confidentiality Exposure</option>
                    <option value="Malware Infection">Malware / Ransomware
Suspicion</option>
                    <option value="Other">Other Security Concern</option>
                </select>

                <label>Perceived Severity:</label>
                <select name="severity">
                    <option value="LOW">Low - Minimal Risk / Inquiry</option>
```

```
                    <option value="MEDIUM">Medium - Single Account / Non-Critical
Device</option>
                    <option value="HIGH">High - Compromised Credentials / Active
Malware</option>
                    <option value="CRITICAL">Critical - Data Breach / System-Wide
Outage</option>
                </select>

                <label>Detailed Description:</label>
                <textarea name="description" rows="4" required
placeholder="Describe what happened..."></textarea>

                <button type="submit">Submit Security Report</button>
            </form>
        </div>

        <!-- SOC Dashboard -->
        <div class="card">
            <h2>SOC Incident Triage Dashboard</h2>
            <table>
                <thead>
                    <tr>
                        <th>Ref ID</th>
                        <th>Reporter</th>
                        <th>Category</th>
                        <th>Severity</th>
                        <th>Status</th>
                        <th>Submitted At</th>
                    </tr>
                </thead>
                <tbody>
                    {% for inc in incidents %}
                    <tr>
                        <td><strong>{{ inc[1] }}</strong></td>
                        <td>{{ inc[2] }}</td>
                        <td>{{ inc[3] }}</td>
                        <td><span class="badge {{ inc[4] }}">{{ inc[4]
}}</span></td>
                        <td><code>{{ inc[6] }}</code></td>
                        <td>{{ inc[7] }}</td>
                    </tr>
                    {% else %}
                    <tr>
                        <td colspan="6" style="text-align: center; color:
#94a3b8;">No security incidents reported yet.</td>
                    </tr>
                    {% endfor %}
                </tbody>
```

```
            </table>
        </div>
    </div>
</body>
</html>
"""

# 3. Web Routes
@app.route('/')
def index():
    conn = sqlite3.connect(DB_FILE)
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM incidents ORDER BY id DESC")
    incidents = cursor.fetchall()
    conn.close()
    return render_template_string(HTML_TEMPLATE, incidents=incidents)

@app.route('/report', methods=['POST'])
def report():
    email = request.form.get('email')
    category = request.form.get('category')
    severity = request.form.get('severity')
    description = request.form.get('description')

    timestamp = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    ref_id = f"INC-{int(datetime.datetime.now().timestamp()) % 100000}"

    conn = sqlite3.connect(DB_FILE)
    cursor = conn.cursor()
    cursor.execute('''
        INSERT INTO incidents (incident_ref, reporter_email, category, severity,
description, created_at)
        VALUES (?, ?, ?, ?, ?, ?)
    ''', (ref_id, email, category, severity, description, timestamp))
    conn.commit()
    conn.close()

    return redirect(url_for('index'))

if __name__ == '__main__':
    print("🚀 Starting Security Incident Reporting Portal on
http://localhost:5000")
    app.run(port=5000, debug=True)
```

5. Installation & Local Setup

Step 1: Prerequisites

Ensure Python 3.x is installed on your machine. Install Flask using terminal:

```shell
pip install flask
```

Step 2: Run Application

Save the source code as `app.py` and run it:

```shell
python app.py
```

Step 3: Access Portal

Open your web browser and navigate to:

```
http://localhost:5000
```

---

6. Incident Response Lifecycle Workflow

When an employee submits a report through the portal, the incident moves through the standard four-stage Incident Response (IR) lifecycle:

1. **Discovery & Intake:** The user detects an anomaly and submits details via the portal form.
2. **Identification & Logging:** The application generates a tracking ID (`INC-XXXXX`), records timestamps, and persists data into SQLite storage.
3. **Triage & Containment:** SOC analysts monitor the dashboard, assess severity, contact the reporter, and apply containment actions.
4. **Resolution & Closure:** Affected systems are restored, post-incident reviews are documented, and the ticket status is updated to `CLOSED`.